Le novità di SQL Server 2025 per gli sviluppatori

Danilo Dominici
Consulente indipendente

ddominici@sqlserverinfo.it

Chi sono

- Consulente indipendente
  - SQL Server (On-Prem/Azure/AWS), PostgreSQL, Power BI
- Microsoft Certified Trainer dal 2000
- 6x Microsoft MVP

ddominici@gmail.com

linkedin.com/in/danilodominici

github.com/ddominici/presentations

25 anni

2014-2020

SQL Server 2025

The AI-ready enterprise database from ground to cloud

AI built-in

Develop modern data applications

Integrate your data with Fabric

Secure by default

Mission critical engine

Connected with Arc

Assisted by Copilots

Benchmark leader

Optimized for latest hardware

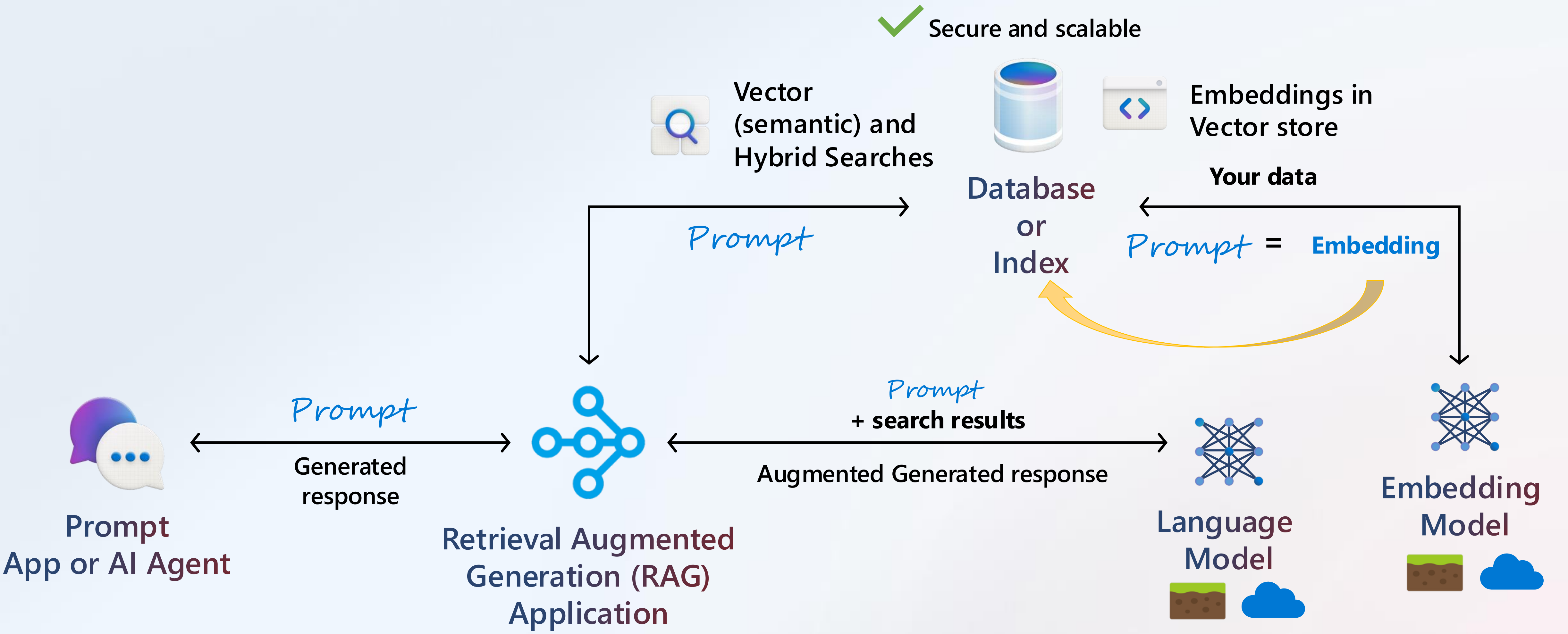
Built for all platforms

Integrato con l'AI

What problems are we trying to solve with AI?

- Smarter searching on your existing text data
- Bring in other documents/text for centralized vector searching
- Provide building blocks - Intelligent assistants, RAG, AI Agents
- Take advantage of AI in a secure and scalable fashion
- Overcome complexity by using the familiar T-SQL language

Vector search with your data



The prompt may contain words *not found* in your data!

What are the features?

Build Agentic RAG patterns *Inside* the engine

NVIDIA ONNX

Vector Store
Native vector data type and DiskANN index*

Model Management
Secure Model definitions ground or cloud

Embeddings built-in
Text Chunking and embedding generation

Simple semantic searching with hybrid
Vector distance (KNN) and Vector search (ANN)*

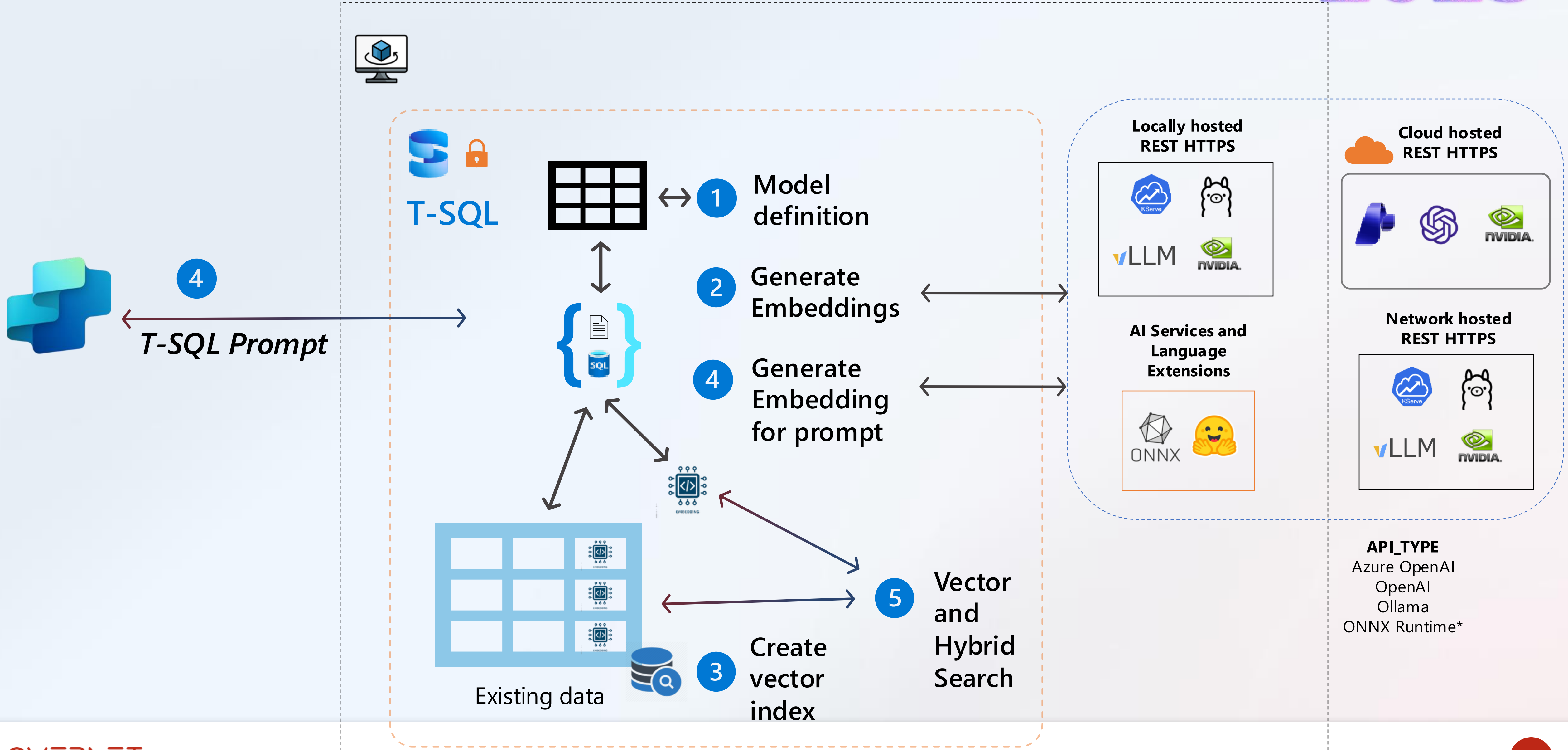
Framework integration
LangChain, Semantic Kernel, EF Core, Agent Framework

*Enabled by PREVIEW_FEATURES

Security, SQL, and AI

- Off by default!
- **You control** *all access* with SQL security
- **You control** AI models ground *or* cloud *isolated*
- Data exchange with AI models is *encrypted*
- Use RLS, TDE, Dynamic Data Masking, and Auditing
- Ledger for *immutable agentic operations* log

SQL Server 2025 Vector Architecture



Scegliere il modello più adatto

- Hugging Face mantiene una leaderboard dei modelli
  - utile per identificare i migliori modelli

Rank (Bor…	Model	Zero-shot	Memory U…	Number of P…	Embedding D…	Max Tokens	Mean (T…	Mean (TaskT…	Bitext …	Classification	Clustering	Instruction R…
1	llama-embed-nemotron-8b	99%	28629	7B	4096	32768	69.46	61.09	**81.72**	73.21	54.35	10.82
2	gemini-embedding-001	99%	Unknown	Unknown	3072	2048	68.37	59.59	79.28	71.82	54.59	5.18
3	Qwen3-Embedding-8B	99%	28866	7B	4096	32768	**70.58**	**61.69**	80.89	**74.00**	**57.65**	10.06
4	Qwen3-Embedding-4B	99%	15341	4B	2560	32768	69.45	60.86	79.36	72.33	57.15	**11.56**
5	Qwen3-Embedding-0.6B	99%	2272	595M	1024	32768	64.34	56.01	72.23	66.83	52.33	5.09
6	gte-Qwen2-7B-instruct	⚠ NA	29040	7B	3584	32768	62.51	55.93	73.92	61.55	52.77	4.94
7	Linq-Embed-Mistral	99%	13563	7B	4096	32768	61.47	54.14	70.34	62.24	50.60	0.94
8	multilingual-e5-large-instruct	99%	1068	560M	1024	514	63.22	55.08	80.13	64.94	50.75	-0.40
9	embeddinggemma-300m	99%	578	307M	768	2048	61.15	54.31	64.40	60.90	51.17	5.61
10	SFR-Embedding-Mistral	96%	13563	7B	4096	32768	60.90	53.92	70.00	60.02	51.84	0.16
11	text-multilingual-embedding-002	99%	Unknown	Unknown	768	2048	62.16	54.25	70.73	64.64	47.84	4.08

Best practices

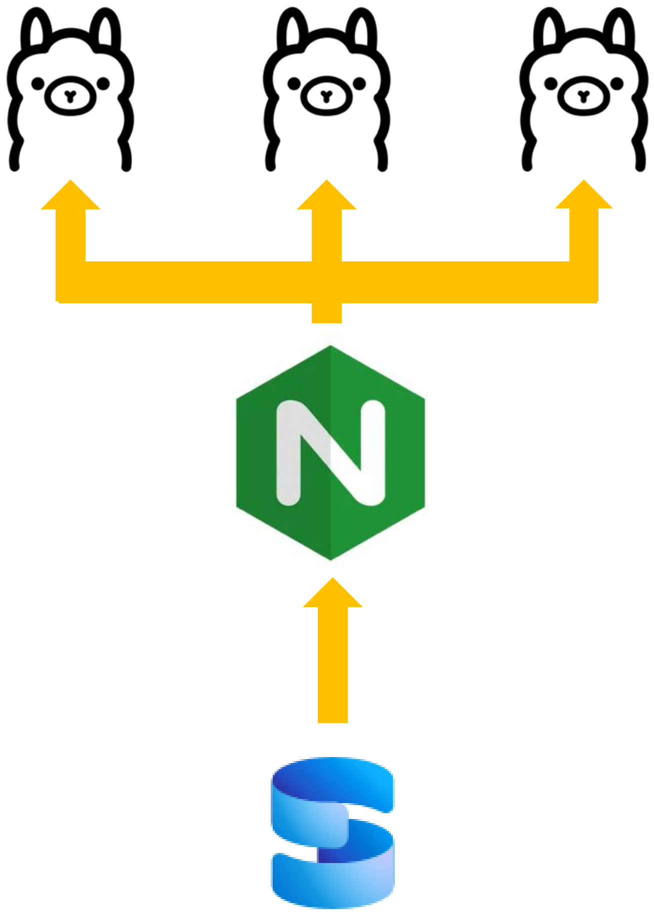
- Usa modelli leggeri
- Esegui inferenze in batch
- Mantieni logging e tracciabilità delle chiamate AI
- Monitora performance e tempi di risposta
- Utilizza cache dei risultati per ridurre le chiamate alle API
  - E quindi costi e latenza
  - Es. Redis

Ollama & performance

- Problema: chiamare un servizio di embedding esterno a SQL Server via REST su HTTPs è limitato dalla banda del backend.
- Con dataset molto grandi i tempi di attesa possono essere molto lunghi

Ollama & performance

- Soluzione: bilanciare le chiamate di generazione degli embedding utilizzando istanze multiple di Ollama bilanciate ad esempio da Nginx.



Demo

SQL Server 2025 e Ollama

Costruito per gli sviluppatori

Built for developers

Develop modern data applications

Data API Builder

REST or GraphQL

There's more…

GitHub Copilot for MSSQL

Microsoft Python Driver for SQL Server

JSON

JSON type, index, and updated T-SQL functions

Change Event Streaming*

Consume log change events

T-SQL

RegEx and other T-SQL functions

REST API

Connect your data to ***any*** REST interface with HTTPS

{ REST }

gpt-oss

* Enabled by PREVIEW_FEATURES

JSON

< SQL Server 2025

varchar/nvarchar column

Any index that supports varchar/nvarchar

Various functions for JSON

SQL Server 2025

json data type – binary storage up to 2GB

.modify operator

json index

+ JSON_OBJECTAGG and JSON_ARRAYAGG

+ JSON_CONTAINS

+ JSON_QUERY WITH ARRAY WRAPPER

Save space with compressed storage
Less I/O and page reads

Demo

JSON

Some T-SQL Love ♥

RegEx functions

PRODUCT()

BASE64 functions

Fuzzy string match functions

CURRENT_DATE returns DATE

SUBSTRING() optional length

UNISTR()

|| string concat operator

DATEADD() supports bigint

The T-SQL RegEx library

LIKE	REGEXP_LIKE
REPLACE	REGEXP_REPLACE
SUBSTR	REGEXP_SUBSTR
CHARINDEX, PATINDEX	REGEXP_INSTR
LEN and REPLACE	REGEXP_COUNT
CHARINDEX, PATINDEX, SUBSTR	REGEXP_MATCHES
CHARINDEX, REPLACE, SUBSTR	REGEXP_SPLIT_TO_TABLE

{ REST }

REST in the engine?

Securely avoid roundtrips to send data to a REST endpoint

Off by default

EXECUTE @returnValue = **sp_invoke_external_rest_endpoint**
[@url =] N'url' → HTTPS local or remote

[, [@payload =] N'request_payload'] → JSON, XML ,TEXT from your data

[, [@headers =] N'http_headers_as_json_array']
[, [@method =] 'GET' | 'POST' | 'PUT' | 'PATCH' | 'DELETE' | 'HEAD']
[, [@timeout =] seconds]
[, [@credential =] credential] → API key, SAS token, managed identity

[, @response OUTPUT] → NVARCHAR but often JSON

[, [@retry_count =] # of retries if there are errors]

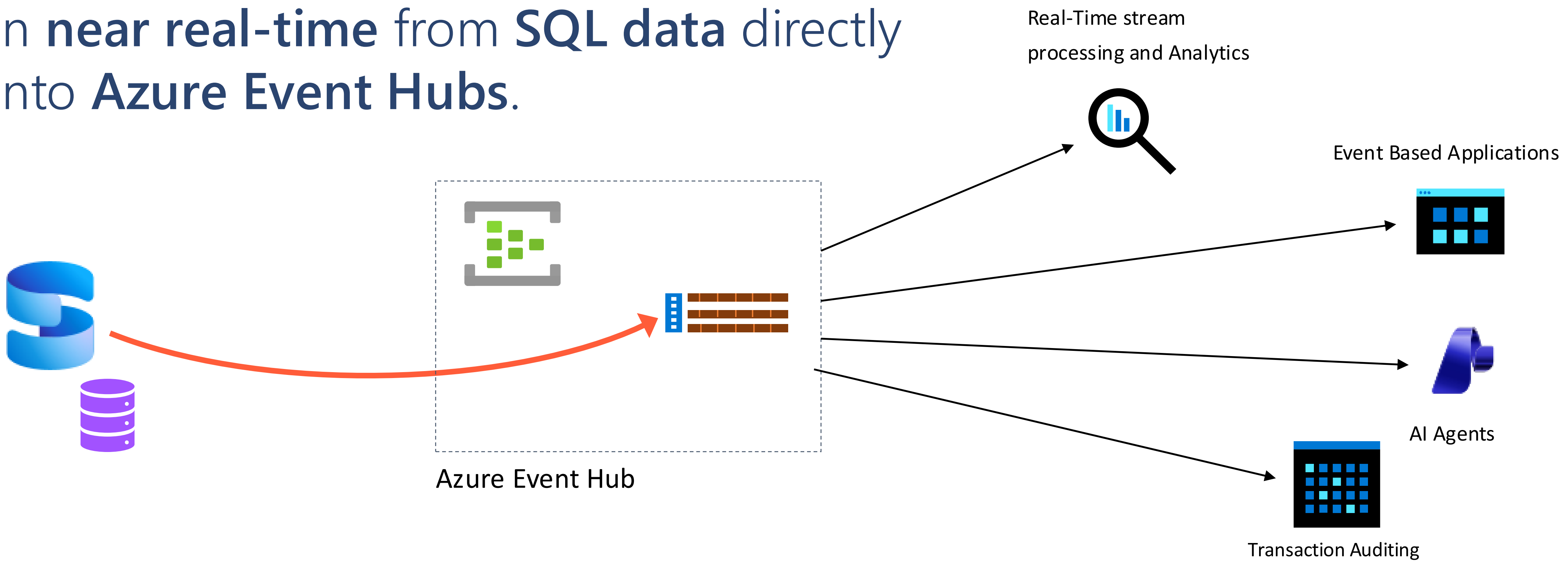
HTTP_EXTERNAL_CONNECTION wait

Demo

Come utilizzare API REST

Change Event Streaming (CES)

CES enables you to stream your data changes in **near real-time** from **SQL data** directly into **Azure Event Hubs**.



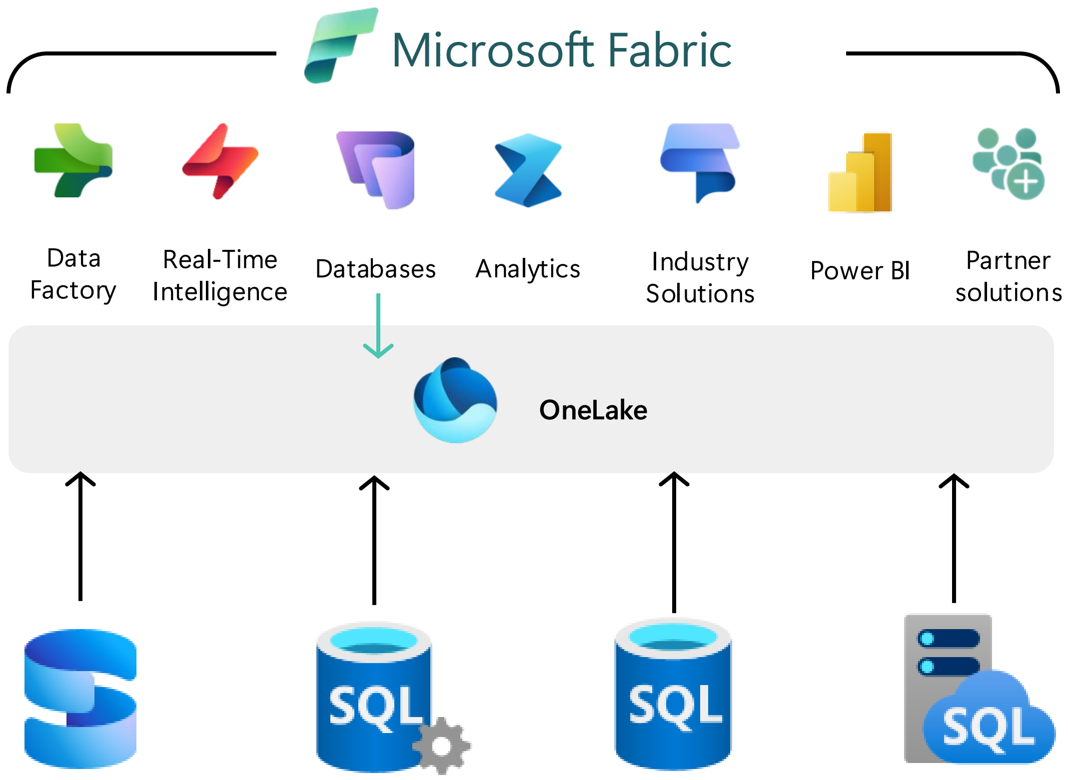
Capturing changes with SQL Server

	Change Tracking (CT)	Change Data Capture (CDC)	Change Event Streaming (CES)
Capture Source	As part of committed transactions (sync)	Committed transactions from transaction log (async). Requires SQL Server Agent	Committed transactions from transaction log (async)
Capture Destination	System table (minimum)	System table (extra logging required)	Event stream (“IOless”)
Data Captured	Row identifier (DDL allowed)	Row identifier and data	Row identifier and data (DDL allowed)
Capture Method	Query system tables and user tables. Multiple destinations	Query system tables. Multiple destinations	Process event stream (“IOless”). Single destination
Availability	Immediate (pull-based)	Near real-time (pull-based)	Near real-time (push-based)
SQL effects	Transaction commit	Transaction log truncation .	Transaction log truncation
Typical Use Cases	Cache invalidation and sync	Data warehousing and auditing	Event-driven architectures – microservices integration, real-time analytics, cache sync, AI Agents

Integrato con Microsoft Fabric

Integrated with Microsoft Fabric

Integrate your operational data with a unified data platform



Mirroring **replicates databases** to **Fabric with zero ETL**

Data is replicated into One Lake and **kept up-to-date in near real-time**

Mirroring **protects operational databases** from **analytical** queries

Compute replication is included with your Fabric capacity **for no cost.**

Free Mirroring Storage for Replicas tiered to Fabric Capacity.

Tanto altro ancora…

SQL Server 2025 editions

The AI-ready enterprise database from ground to cloud

No pricing or licensing changes

Express
Free, entry-level database for small AI, web and mobile apps

Feature highlights
- 50 GB maximum relational databases size NEW
- Up to 4 cores of CPU
- Up to 1410 MBs of memory
- Built-in vector search and semantic search NEW
- Native JSON type support NEW
- External REST endpoint invocation NEW
- Change event streaming NEW
- Microsoft Entra ID authentication NEW
- Mirroring in Fabric NEW

Standard
Full featured database with extensive scale for mid-tier applications

Feature highlights
+ Express features

- Up to 32 cores of CPU NEW
- Up to 256 GB of memory NEW
- Optimized locking NEW
- Resource Governor NEW
- ZSTD Backup Compression Algorithm NEW
- Basic Availability Groups
- Azure SQL Managed Instance link
- Accelerated Database Recovery
- Backups to S3-compatible object storage
- 524 PB maximum relational databases size

Enterprise
Highest performance and scaling for business-critical workloads

Feature highlights
+ Express and Standard features

- Highest core count supported
- Highest memory supported
- Built-in Intelligent Query Processing
- Online and Resumable Index Operations
- Availability Groups: including distributed and contained
- Query Store for readable secondaries
- Persisted Statistics for Secondary Replicas (Query store)

Developer
Free to use for dev/test in non-production environments

Standard Developer NEW

Enterprise Developer

Improving Application Concurrency and Availability

Optimized Locking

Eliminate
lock escalation

Lock After
Qualification (LAQ)

Requires
ADR,RCSI,
and db option

ABORT_QUERY_EXECUTION query hint

Abort future
query execution

Stop rogue queries
from harming other users

Query Store Hint

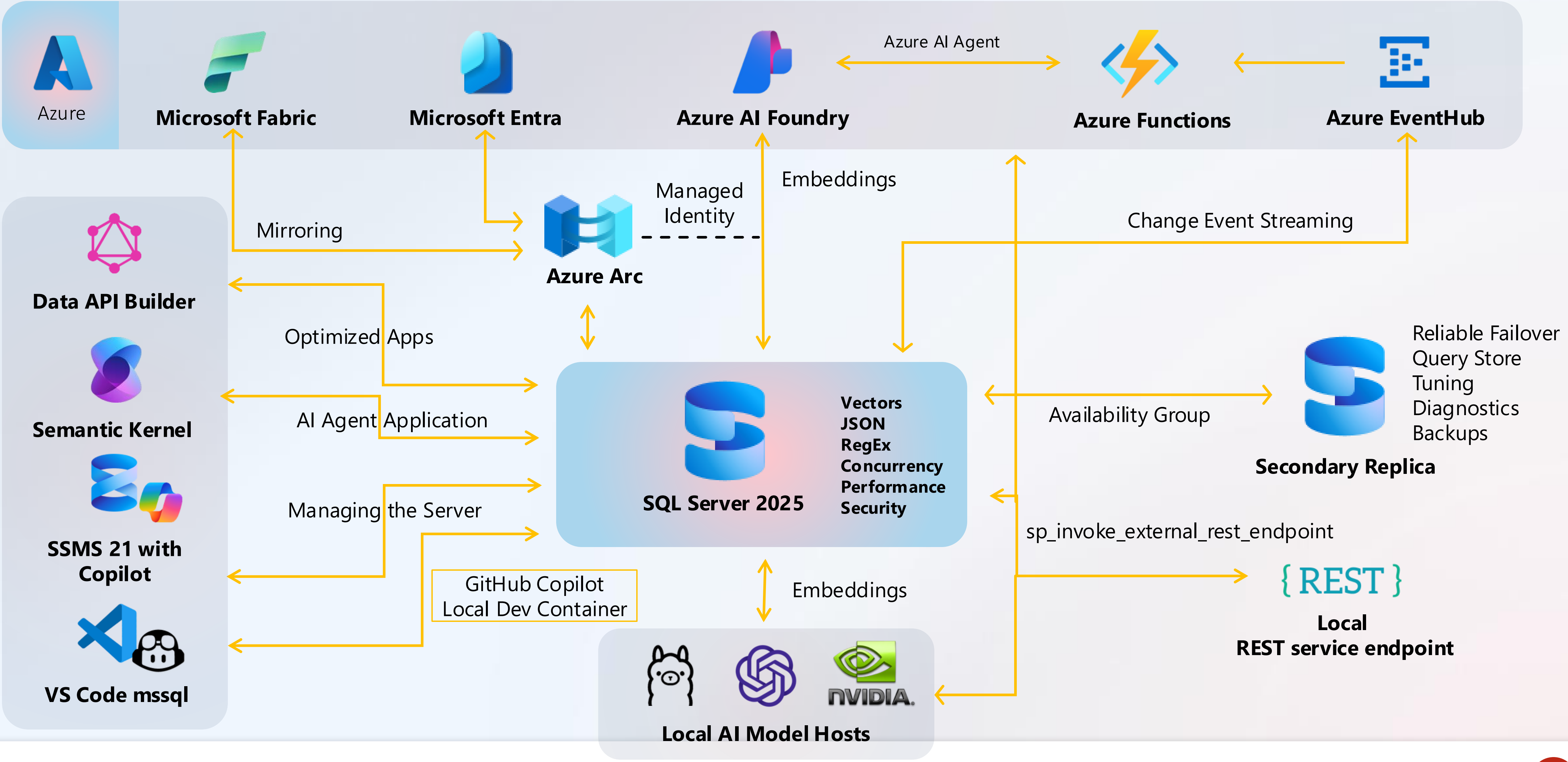
Tempdb Resource Governance

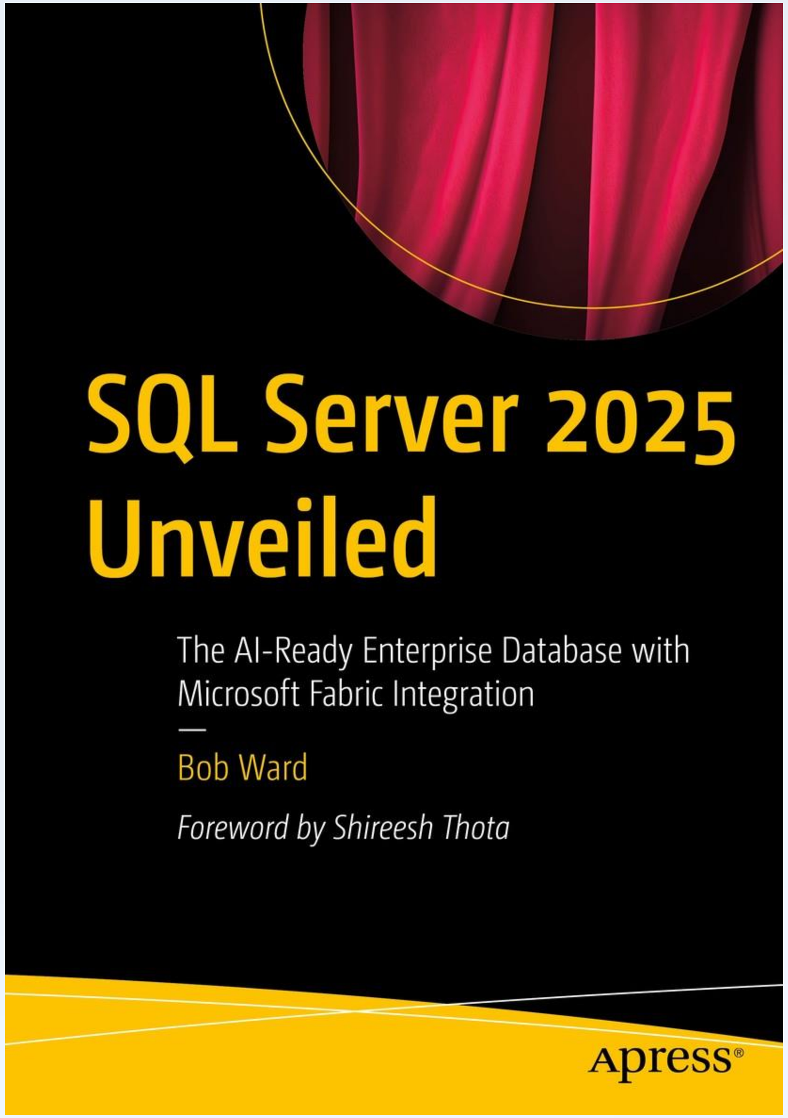
Tempdb space usage can
grow out of control

Need to control and
isolate users

Flexibility to change
policy over time.
Fixed size or %

SQL Server 2025: visione di insieme





aka.ms/sql2025book

Cosa fare ora?

Controllare il blog di MS
aka.ms/sqlserver2025blog

Guardare i video (MS)
aka.ms/Build/sql2025mechanics
aka.ms/SQL2025videos

Leggere la documentazione!
aka.ms/sqlserver2025docs

Scaricarlo e provarlo
aka.ms/getsqlserver

Scaricare le demo (MS)
aka.ms/sqlserver2025demos

Scaricare SSMS 22
aka.ms/ssms

Grazie !